Лекция 4. Паросочетания: алгоритмы Куна и Эдмондса.

1. Введение

Паросочетание в графе – это множество рёбер, не имеющих общих вершин. Размер паросочетания – число рёбер в нём. Паросочетание называется **максимальным**, если оно содержит наибольшее возможное число рёбер.

Классическая задача: найти максимальное паросочетание в заданном графе. Для её решения используются **дополняющие пути** – чередующиеся пути, у которых первая и последняя вершины свободны (не входят в текущее паросочетание), а рёбра чередуются: не из паросочетания, из паросочетания, не из паросочетания и т.д.

Теорема Бержа: паросочетание максимально тогда и только тогда, когда не существует дополняющего пути.

Эта теорема даёт общую стратегию: начинаем с пустого паросочетания, пока находим дополняющий путь – увеличиваем паросочетание, инвертируя рёбра на пути. Разные алгоритмы отличаются способом поиска дополняющих путей.

2. Алгоритм Куна для двудольных графов

Если граф **двудольный** (вершины можно разбить на две доли L и R, и все рёбра идут только между долями), задача упрощается. Алгоритм Куна находит максимальное паросочетание за $O(V \cdot E)$ и очень прост в реализации.

2.1 Идея

Будем пытаться найти дополняющий путь из каждой свободной вершины левой доли. Поиск ведётся в глубину с возвратами. Чтобы не зацикливаться, в каждой фазе поиска помечаем посещённые вершины левой доли.

2.2 Псевдокод

Обозначения:

- n – число вершин в левой доле, m – в правой.
- $graph[v]$ – список смежных вершин из правой доли для вершины v левой доли.
- $matchR[u]$ – для вершины u правой доли храним номер вершины левой доли, с которой она спарена (или -1, если свободна).
- $used[v]$ – флаг посещения в текущем DFS.

```
function kuhn_max_matching(graph, n, m):
   matchR = [-1] * m          # сначала все вершины правой доли свободны
```

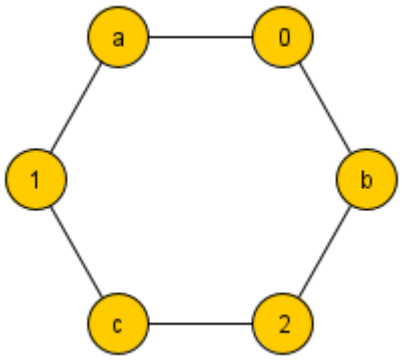
```
    result = 0

    for v in range(n):
        used = [False] * n     # обнуляем пометки для каждой новой вершины v
        if dfs(v, graph, used, matchR):
            result += 1

    return result, matchR

function dfs(v, graph, used, matchR):
    if used[v]:
        return False
    used[v] = True

    for u in graph[v]:
        # Если u свободна или удалось перекинуть пару из matchR[u]
        if matchR[u] == -1 or dfs(matchR[u], graph, used, matchR):
            matchR[u] = v
            return True
    return False
```

2.3 Пояснения

- Внешний цикл перебирает все вершины левой доли. Если вершина v свободна (в $matchR$ ещё нет пары для неё, но мы не храним отдельно $matchL$, просто пытаемся найти дополняющий путь, начиная с v), то запускаем DFS.
- В DFS пытаемся пройти по всем соседям u. Если u не занята, то сразу закрепляем ребро (v, u) (становится частью паросочетания). Если u занята вершиной $w = matchR[u]$, то рекурсивно пытаемся найти другой путь для w (т.е. перекинуть её на другое ребро). Если это удаётся, то перекидываем и v соединяем с u.
- Пометки $used$ нужны, чтобы не ходить по одним и тем же вершинам левой доли в одной фазе поиска и не зацикливаться.

2.4 Пример

Рассмотрим граф:



Начальное паросочетание пусто.

1. Из вершины 0: идём по 0-a ⇒ a свободна ⇒ паросочетание $\{(0, a)\}$.

2. Из вершины 1: идём по 1-a ⇒ a занята 0, пытаемся перекинуть 0. Из 0 идём по 0-b (a уже пробовали) ⇒ b свободна ⇒ перекидываем 0 на b, тогда 1 получает a. Паросочетание $\{(0, b), (1, a)\}$.

3. Из вершины 2: идём по 2-b ⇒ b занята 0, пытаемся перекинуть 0. Из 0 идём по 0-a (b уже было) ⇒ a занята 1, пытаемся перекинуть 1. Из 1 идём по 1-c ⇒ c свободна ⇒ перекидываем 1 на c, тогда 0 возвращается на a? Но нужно аккуратно. В итоге после рекурсии получим паросочетание $\{(0, a), (1, c), (2, b)\}$. Размер 3 – максимально.

2.5 Сложность

Каждый вызов DFS может просмотреть $O(E)$ рёбер, и таких вызовов $O(V)$. Итоговая сложность $O(V \cdot E)$. На практике часто работает быстрее благодаря раннему выходу.

3. Проблема общих (не двудольных) графов

В не двудольных графах могут быть циклы нечётной длины. При попытке применить тот же подход (DFS) можно столкнуться с ситуацией, когда в вершину можно прийти двумя путями разной чётности (один длиннее другого на нечётное число). Это приводит к тому, что простой DFS может пропустить дополняющий путь, даже если он существует.

Пример: граф с нечётным циклом (треугольник) и висячим ребром. Дополняющий путь есть, но DFS из-за пометок может не найти его, так как в какой-то момент зайдёт в вершину с одной чётностью, а потом не сможет использовать её с другой.

Такая ситуация называется **цветком**. Формально: **цветок** – это нечётный цикл, в котором все вершины, кроме одной (основания), имеют чередующиеся рёбра, и в цикле есть ровно одно ребро паросочетания, ведущее к свободной вершине (стеблю). Цветок мешает поиску дополняющего пути, но его можно **сжать** в одну вершину, что сохраняет существование дополняющего пути в графе.

4. Алгоритм Эдмондса (паросочетания в общих графах)

Алгоритм Эдмондса, также известный как **алгоритм цветков**, обобщает идею дополняющих путей на произвольные графы. Он систематически ищет дополняющие пути, а при обнаружении цветка сжимает его и продолжает поиск в сжатом графе. Если в сжатом графе найден дополняющий путь, его можно развернуть в путь в исходном графе.

4.1 Основные определения

Чередующийся лес – лес, построенный из свободных вершин чередующимися путями.

Вершины леса делятся на **чётные** (достижимые путём чётной длины) и **нечётные** (нечётной длины). Корни (свободные вершины) – чётные.

Если обнаруживается ребро между двумя чётными вершинами в разных деревьях, то найден дополняющий путь (чётно-чётное ребро не из паросочетания). Если ребро между двумя чётными вершинами в одном дереве – возникает **цветок** (нечётный цикл). Ребро между чётной и нечётной вершиной игнорируется (ведёт в паросочетании).

4.2 Сжатие цветка

Пусть найден цветок – нечётный цикл, состоящий из чётных вершин, соединённых ребром не из паросочетания, и чередующихся рёбер цикла. Все вершины цветка сжимаются в одну **супер-вершину**. При этом рёбра, инцидентные любой вершине цветка, теперь считаются инцидентными супер-вершине. Паросочетание внутри цикла фиксируется так, чтобы супер-вершина была чётной (или нечётной в зависимости от контекста).

Лемма: в графе существует дополняющий путь тогда и только тогда, когда он существует в сжатом графе.

4.3 Поиск дополняющего пути с обнаружением цветков

Алгоритм использует BFS/DFS с поддержкой меток чётности и системы непересекающихся множеств для сжатых цветков. Упрощённо его можно описать так:

1. Строим чередующийся лес из всех свободных вершин.
2. Рассматриваем рёбра, не входящие в лес, в порядке обхода:

- Если ребро соединяет две чётные вершины из разных деревьев – найден дополняющий путь (восстанавливаем его через предков и сжатые цветки).
- Если ребро соединяет две чётные вершины из одного дерева – найден цветок. Определяем его вершины, сжимаем их в одну супер-вершину и продолжаем построение леса с этой супер-вершиной (она будет чётной).

3. Если дополняющий путь найден, разворачиваем его, проходя через сжатые цветки и инвертируя рёбра.

4.4 Псевдокод (концептуальный)

Приведём упрощённую версию алгоритма Эдмондса без деталей работы со структурами данных.

```
# Структура для вершины
class Node:
   parent = None        # родитель в чередующемся дереве
   label = None         # 'even' / 'odd' / None
   base = None          # для сжатых цветков: представитель супер-вершины
   match = None         # вершина, с которой текущая в паросочетании

function find_augmenting_path(root, graph, match):
   # root - свободная вершина, начинаем с неё строить дерево
   queue = [root]
   root.label = 'even'
   root.parent = None
   # BFS
   while queue not empty:
      v = queue.pop()
      for u in graph[v]:
         if u.label is None:
            # ребро из v в u - ещё не обработанное
            if u.match is None:
               # u свободна? Это дополняющий путь? Но u может быть в другом дереве.
               # На самом деле дополняющий путь обнаруживается только через чётно-чётные рёбра.
               # Пропустим сложности.
            elif u.label == 'even' and u != v.parent:
               # Найдено ребро между двумя чётными вершинами - возможный цветок
               blossom = find_blossom(v, u)
               contract_blossom(blossom)
               # продолжать с супер-вершиной
            # ... остальные случаи
   # ...
```

Более реалистичный псевдокод занял бы несколько страниц.

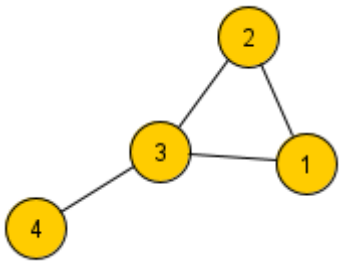
4.5 Разжатие цветка

Когда дополняющий путь найден в сжатом графе, нужно восстановить его в исходном. Для этого:

- Если путь проходит через супер-вершину, то внутри цветка можно провести чередующийся путь от входа до выхода, используя рёбра цветка (всегда есть два варианта, один из которых даст чередование).
- После этого инвертируем рёбра на дополняющем пути, как обычно.

4.6 Пример

Рассмотрим граф с нечётным циклом (треугольник) и дополнительной вершиной. Например:



Паросочетание: {1-2} (занято). Начинаем из свободной вершины 4. Построим чередующееся дерево: 4 (чётная) ⇒ ребро 4-3 (не в паросочетании) ⇒ 3 (нечётная) ⇒ из 3 ребро паросочетания 3-? нет, т.к. 3 свободна? Но 3 не в паросочетании, тогда 3 свободна? На самом деле паросочетание {1-2}, значит 3 свободна. Тогда из 4-3 – это прямо дополняющий путь? Да, 4 свободна, 3 свободна, ребро не из паросочетания – путь длины 1. Значит, дополняющий путь есть. Но если бы паросочетание было иначе, например {2-3} и свободны 1 и 4, то из 4: 4-3 (3 занята) ⇒ идём по ребру паросочетания 3-2 (2 нечётная) ⇒ из 2 ищем свободное ребро: 2-1 (1 свободна) – путь 4-3-2-1, дополняющий. В этом примере нет цветка. Нужен более хитрый граф, где DFS зацикливается. Но для иллюстрации сжатия нужен реальный цветок, например пятиугольник с чередованием. Приведём его в тексте словесно.

4.7 Сложность

- Наивная реализация алгоритма Эдмондса работает за $O(V^4)$.
- С использованием эффективных структур данных (система непересекающихся множеств, BFS) можно достичь $O(V^3)$.
- Существуют более современные алгоритмы, но для общих графов часто используют готовые реализации Эдмондса.

5. Сравнение алгоритмов

Характеристика	**Алгоритм Куна (двудольные)**	**Алгоритм Эдмондса (общие)**
Область применения	Только двудольные графы	Любые неориентированные графы
Время работы	$O(V \cdot E)$	$O(V^3)$ или $O(V^4)$
Память	$O(V + E)$	$O(V + E)$
Сложность реализации	Очень простая	Сложная (требует обработки цветков)
Использование на практике	Очень часто	Редко, только при необходимости